

# Deep Dive: CUDA optimisation of exoplanet transit detection (CETRA)

Leigh Smith - University of Cambridge  
[lsmith@ast.cam.ac.uk](mailto:lsmith@ast.cam.ac.uk)

# Background

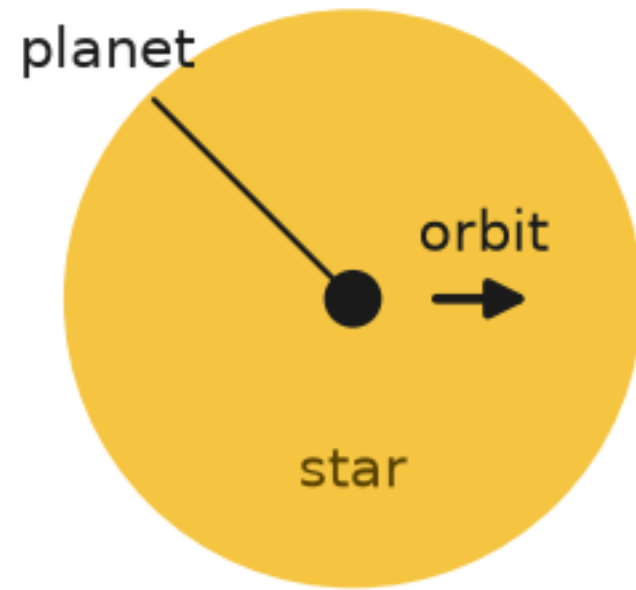
Research IT system manager  
Centre for Astrophysics Research  
University of Hertfordshire (2015  $\Rightarrow$  2017)

Senior research associate  
Institute of Astronomy  
University of Cambridge (2017  $\Rightarrow$  present)

Software developer for the ESA PLATO Exoplanet Analysis Pipeline

# Exoplanet transits

# Exoplanet transit primer



A Jupiter-sized transit

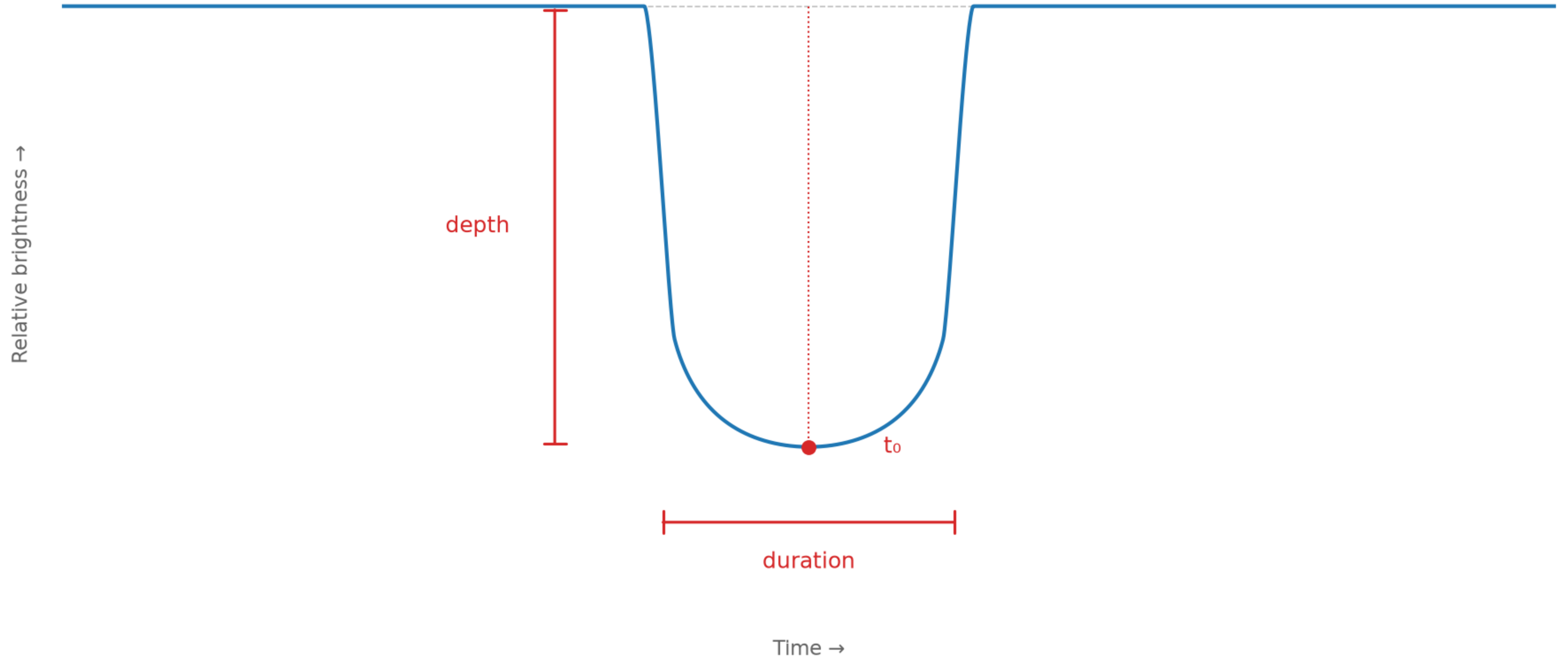


A small Earth-like transit

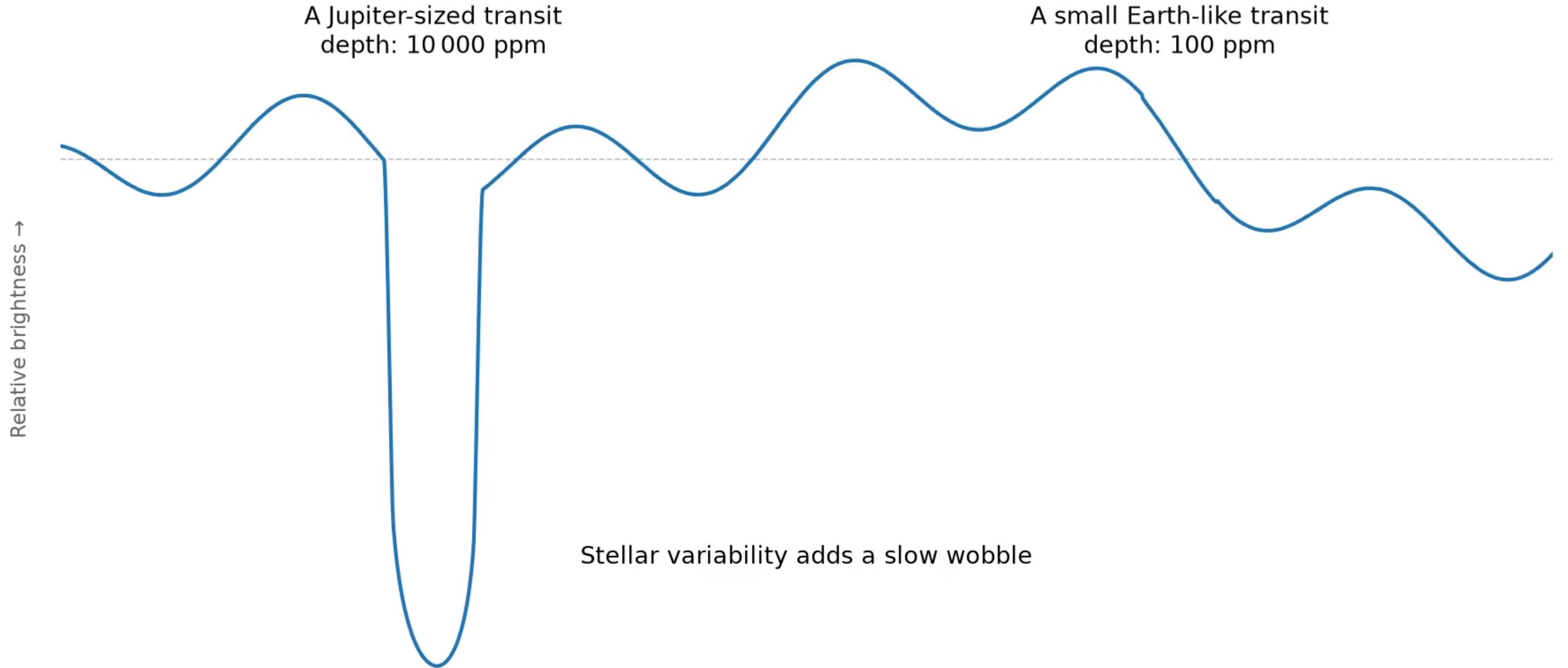
Relative brightness →

Time →

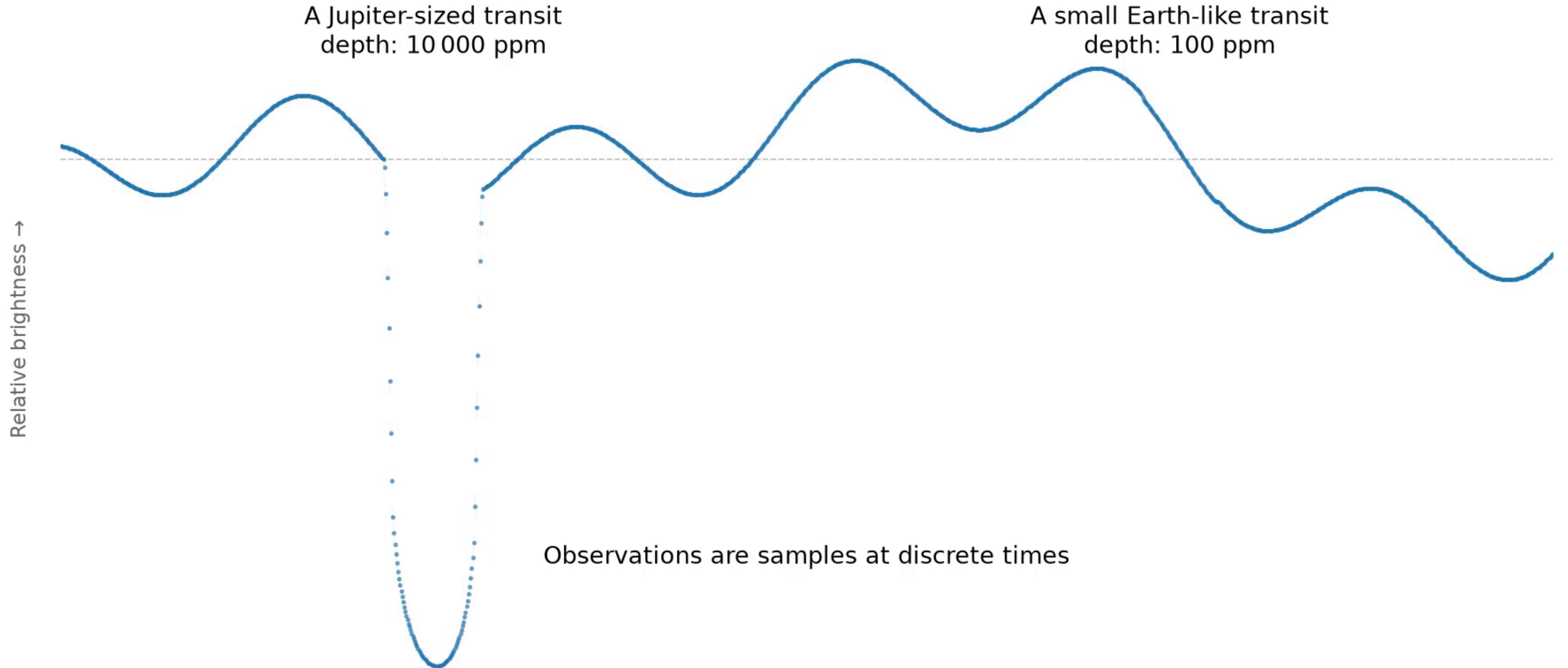
# Exoplanet transit primer



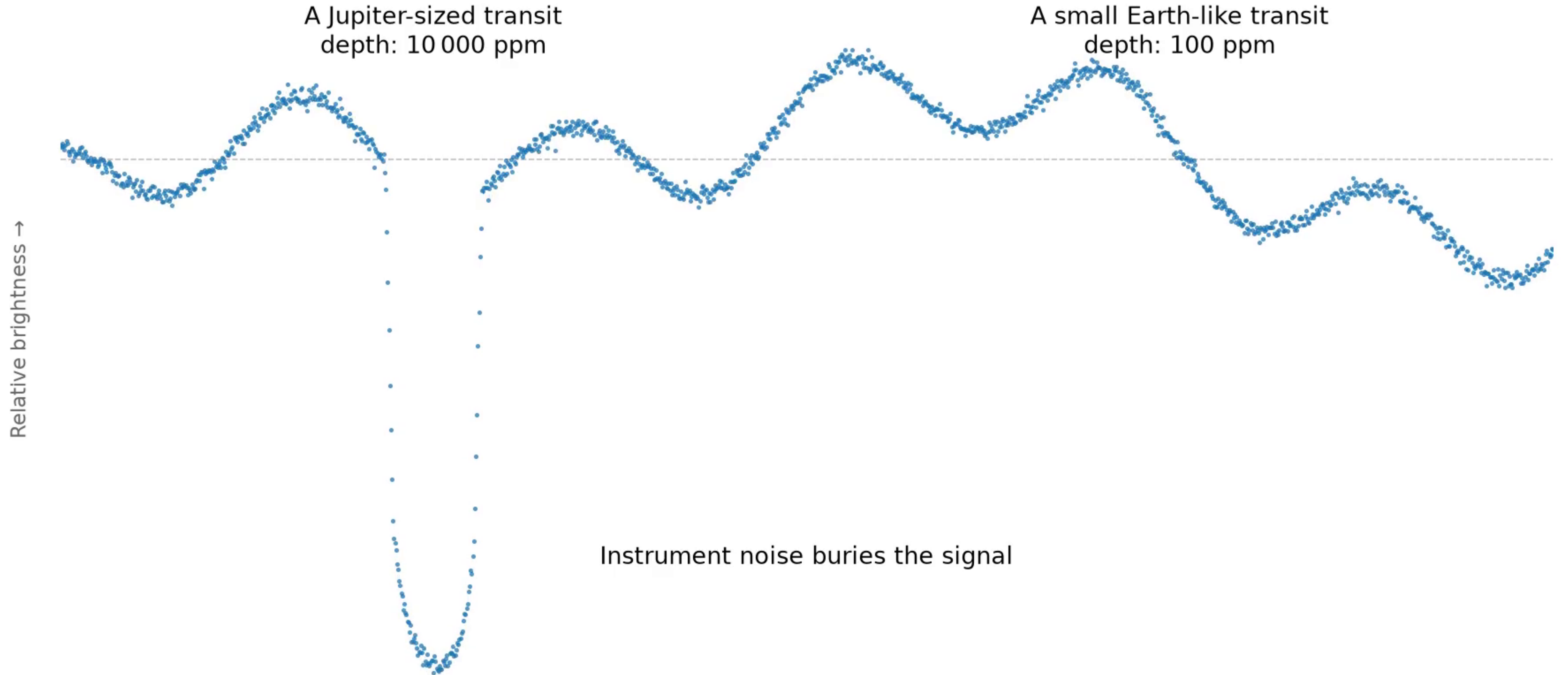
# Exoplanet transit primer



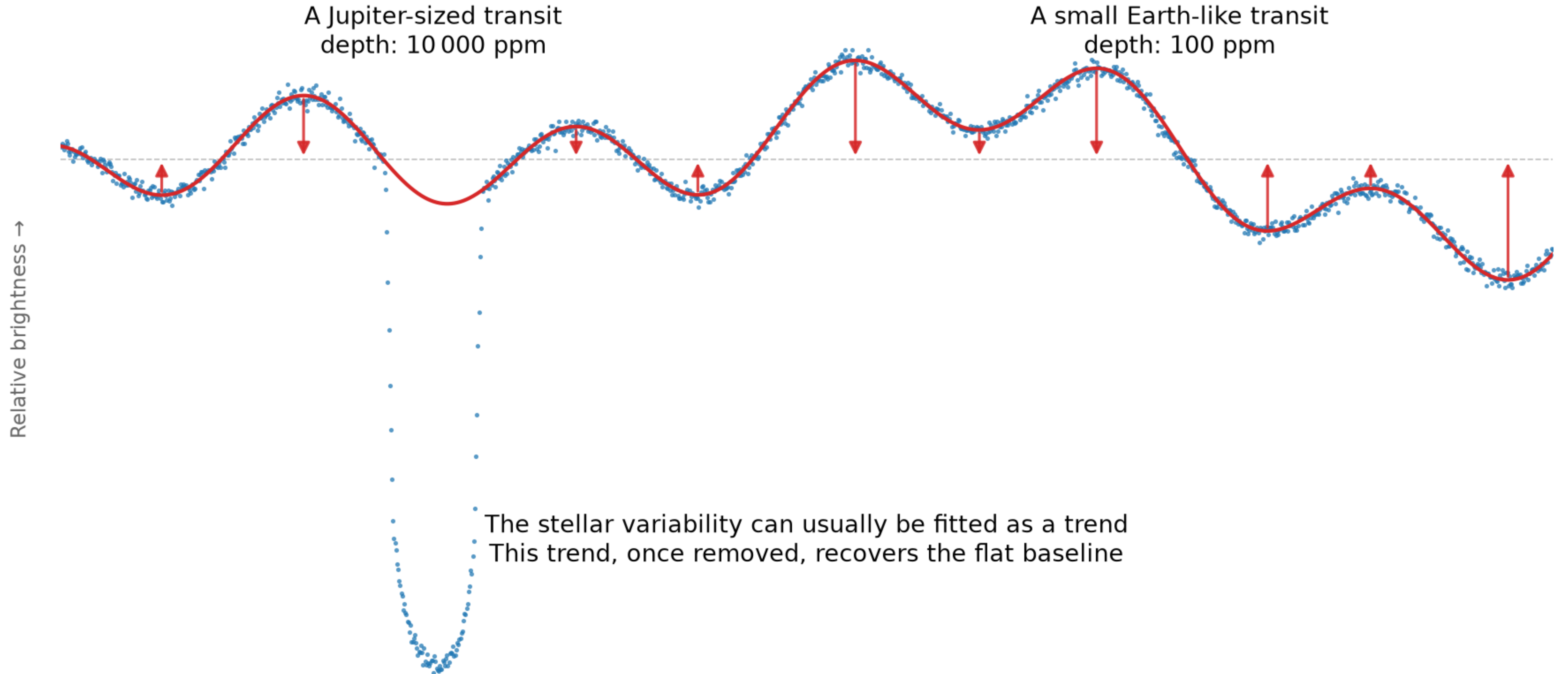
# Exoplanet transit primer



# Exoplanet transit primer



# Exoplanet transit primer

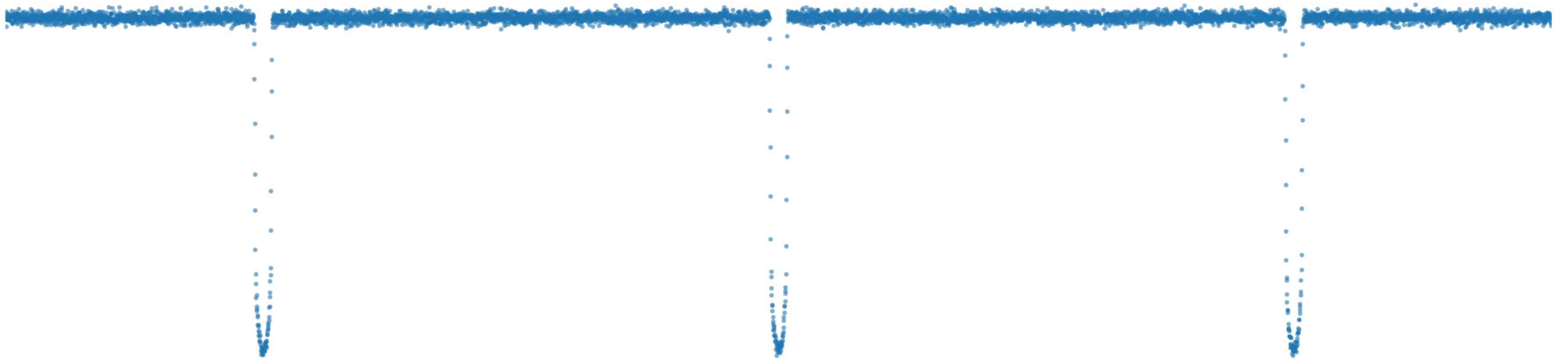


# Exoplanet transit surveys

| Mission | Cadence                | Baseline          | Targets            |
|---------|------------------------|-------------------|--------------------|
| Kepler  | $\approx 30$ min       | $\approx 4$ years | $\approx 200\,000$ |
| TESS    | $\approx 2$ min        | $\approx 1$ month | $\approx 200\,000$ |
| PLATO   | $\approx 25 / 600$ sec | $\approx 2$ years | $\approx 260\,000$ |

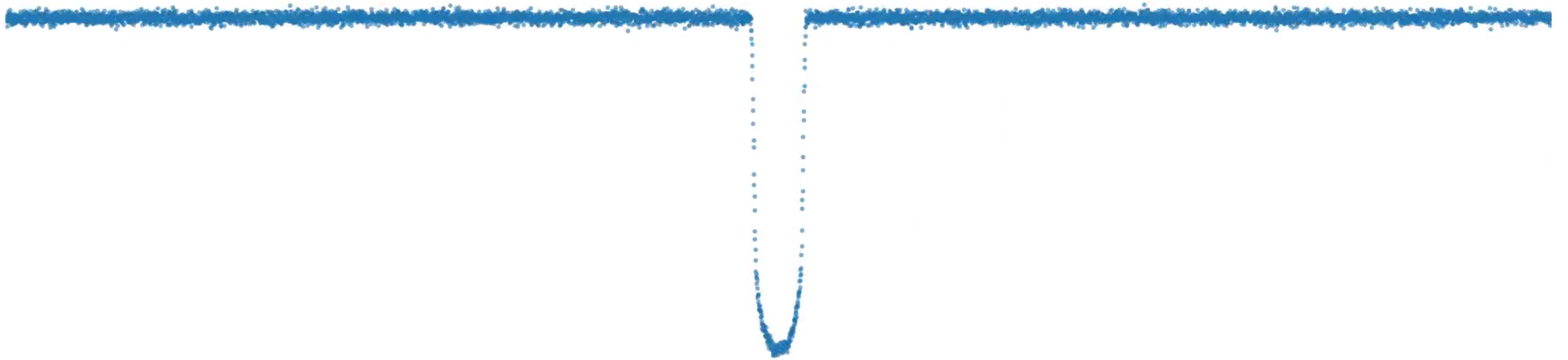
# Transit detection

# Exoplanet transit searches



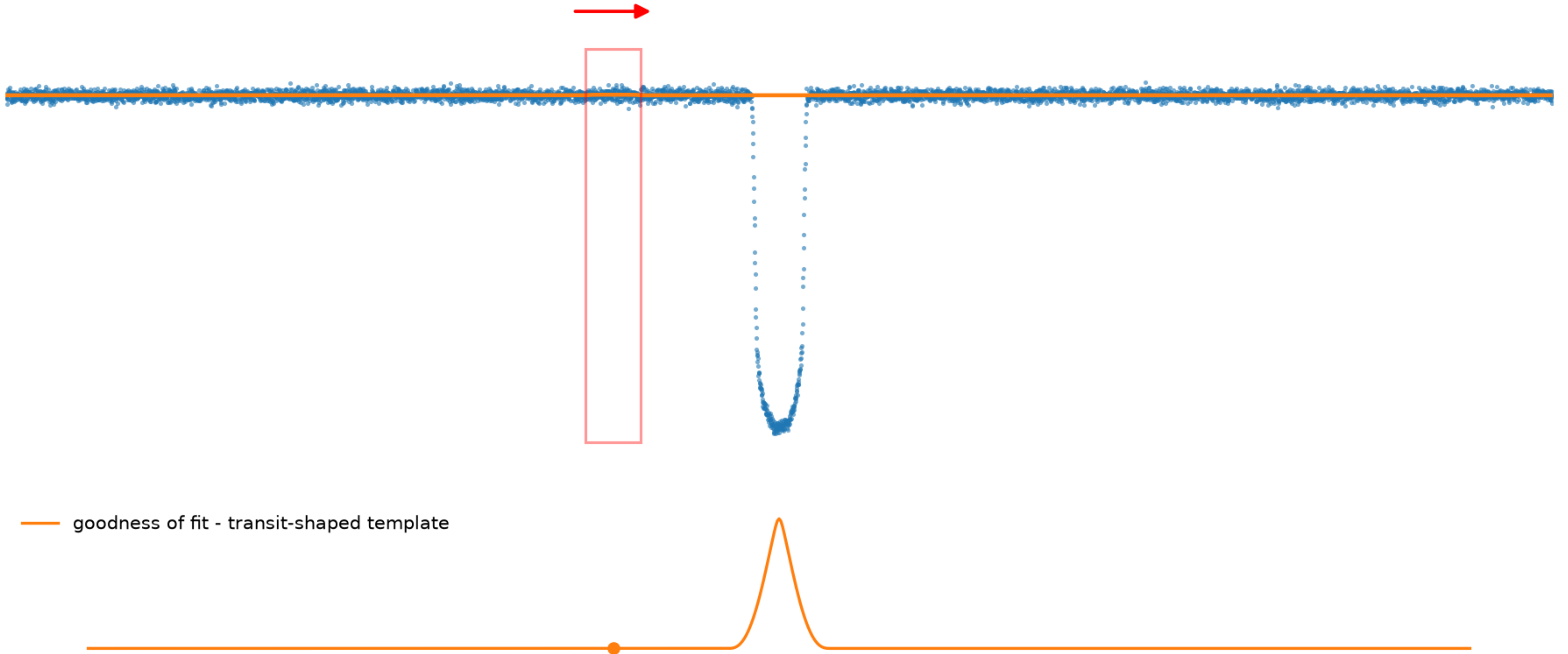
Raw light curve, several transits

# Exoplanet transit searches



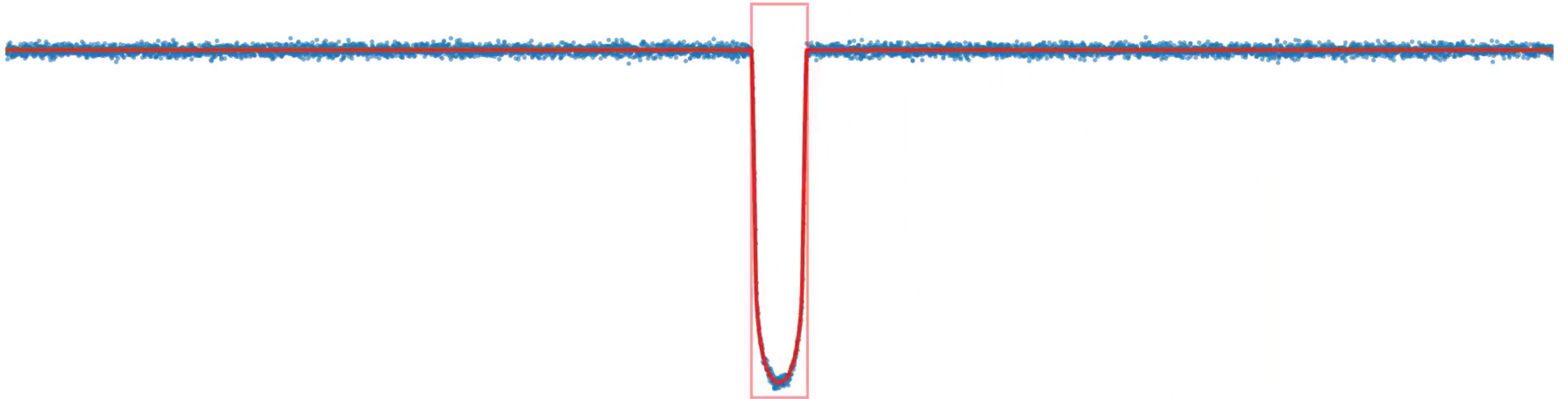
Phase-fold at the trial period

# Exoplanet transit searches



Slide a transit-shaped template, fitting the depth at each position

# Exoplanet transit searches



— goodness of fit - transit-shaped template



Best fit: minimum residuals found

# Exoplanet transit searches

**BLS** (Box Least-Squares) and **TLS** (Transit Least-Squares)

Both search a light curve for periodic dips by testing many combinations of period, duration, and transit epoch. They both run on CPUs\*.

**BLS:** Kovács, Zucker & Mazeh (2002), A&A, 391, 369  
fits a simple box-shaped dip — fast, but crude

**TLS:** Hippke & Heller (2019), A&A, 623, A39  
fits a realistic transit shape — more sensitive, but costlier per test

Both are exhaustive grid searches: cost scales with how many combinations must be tried

\* there is a GPU implementation of BLS, but BLS is less sensitive than TLS

# Exoplanet transit searches

Cost of one full transit detection pass with TLS

TESS  
prime

0.3

0.00 0.05 0.10 0.15 0.20 0.25 0.30 0.35 0.40

CPU core-years

# Exoplanet transit searches

Cost of one full transit detection pass with TLS

TESS  
prime

0.3

Kepler  
DR25

84.0

0 20 40 60 80 100

CPU core-years



# Exoplanet transit searches

Cost of one full transit detection pass with TLS

TESS  
prime

0.3

Kepler  
DR25

84.0

PLATO  
primary

381.0

0 50 100 150 200 250 300 350 400

CPU core-years



# Exoplanet transit searches

TLS: three technical deficiencies

- **Depth-uncertainty blindness** — formally sub-optimal (uses arithmetic, not inverse-variance-weighted mean); only bites for systematic, not random, depth-uncertainty variation
- **Coarse  $t_0$  sampling** — worst at long periods, exactly where PLATO's science case sits
- **Non-robust data-gap handling** — same long-period sensitivity: fewer transits observed means each one, and any gap distortion, carries more weight

# Exoplanet transit searches

- **Coarse  $t_0$  sampling**



long cadence, long period phase-folded transit  
standard  $t_0$  grid spacing is 1% of the transit window duration  
but the TLS search window moves by a minimum of one data element at a time  
meaning in such cases the functional grid is coarser than specified

# Exoplanet transit searches

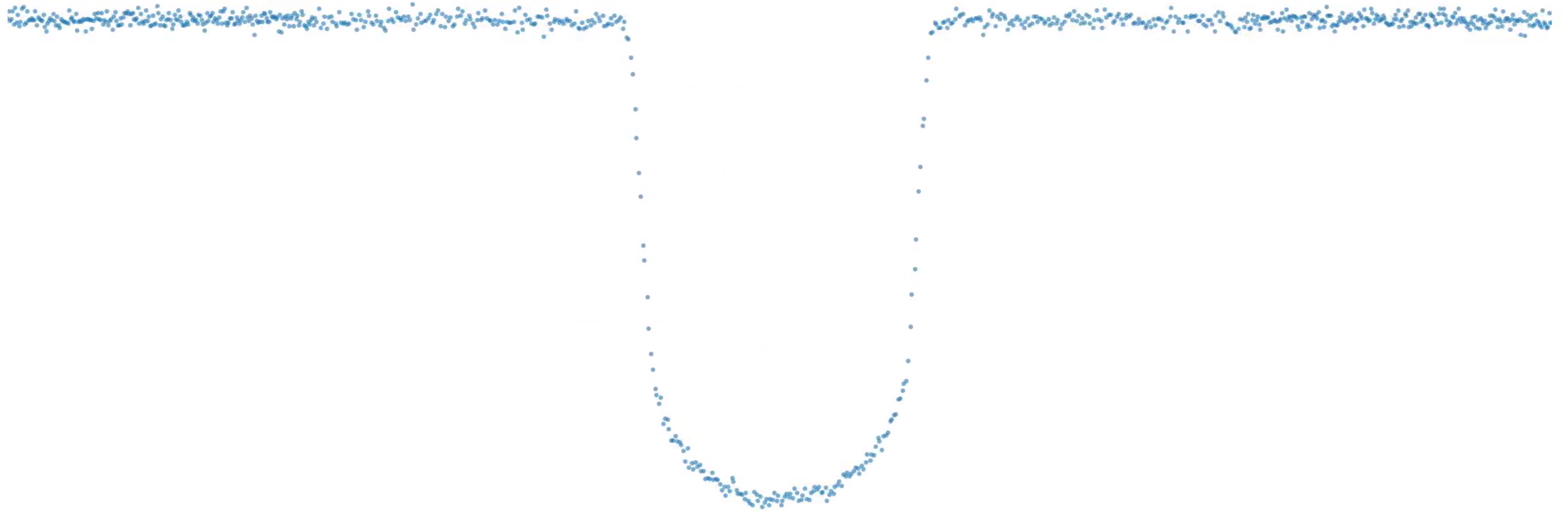
- **Non-robust data-gap handling**



An observation gap falls on one of the transits

# Exoplanet transit searches

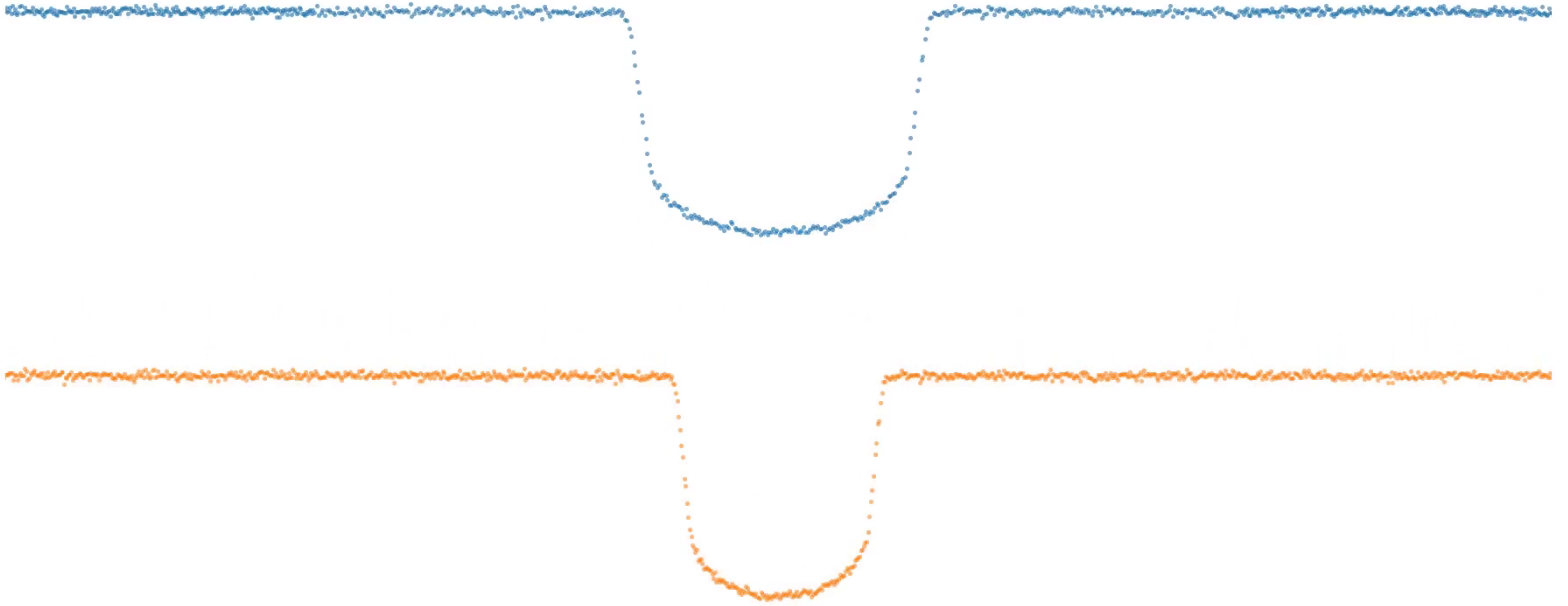
- **Non-robust data-gap handling**



Phase-fold the light curve

# Exoplanet transit searches

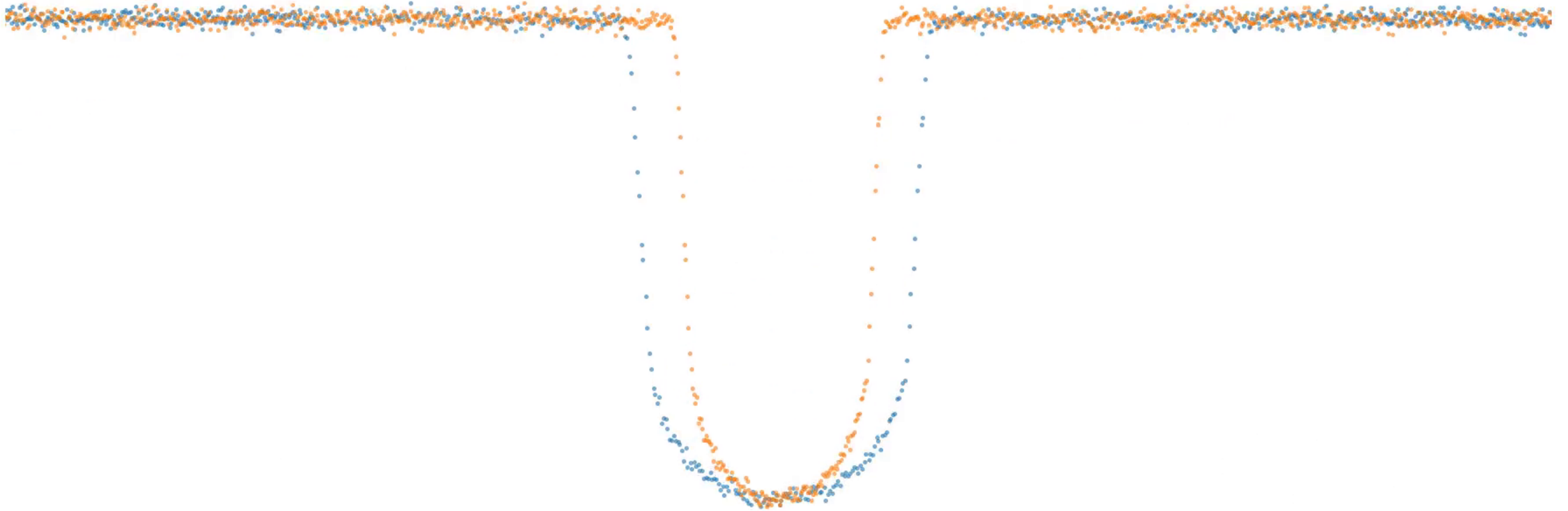
- **Non-robust data-gap handling**



Same points, by array element instead of phase (how TLS works)

# Exoplanet transit searches

- **Non-robust data-gap handling**



The transit appears narrower than in reality — data gaps can distort transit shapes when not treated carefully

**CETRA**

# A new approach — CETRA

## Why a straight port doesn't work

- The key issue isn't implementation, it's the algorithm's structure
- TLS phase-folds the light curve first, then searches — but the fold depends on the trial period
- That means **re-sorting the entire light curve for every trial period**
- On a GPU, that repeated cost compounds badly, especially at high cadence — true regardless of how fast any single sort is
- The right approach isn't "port TLS to CUDA," it's rethinking the algorithm for the architecture

# A new approach — CETRA

## Linear search, then periodic search

- Split the problem into two phases, instead of the TLS sort-per-period approach
- **Linear search:** for every duration  $\times t_0$  window, compute the best-fit depth and likelihood — independent of period, and extremely cheap (seconds or less)
- **Periodic search:** phase-fold those already-computed results across a grid of trial periods, and combine — still the dominant cost, but significantly cheaper than a per-period re-sort at high cadence
- The expensive part now scales with the compact linear-search output, not the raw light curve

# CETRA — Linear search

## Overview

1. Produce grid of [duration,  $t_0$ ], defines transit windows
2. For each, find max-likelihood depth given transit model
3. For each, evaluate likelihood of depth-scaled transit model
4. For each, record depth, variance and likelihood

# CETRA – Linear search

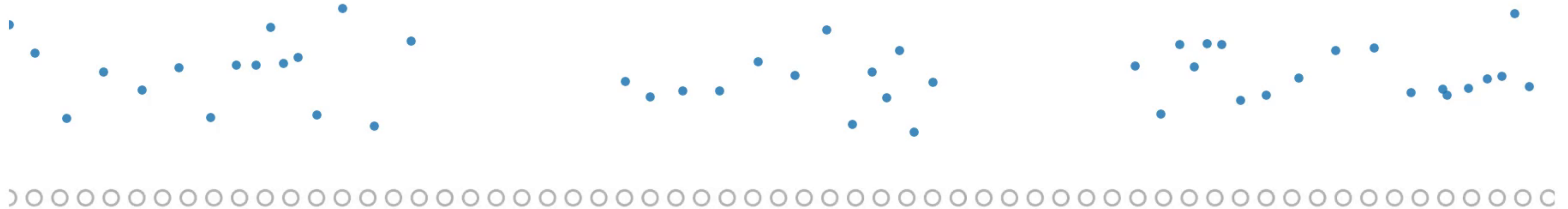
## GPU Optimisation: Light curve resampling



irregularly sampled light curve with two data gaps

# CETRA — Linear search

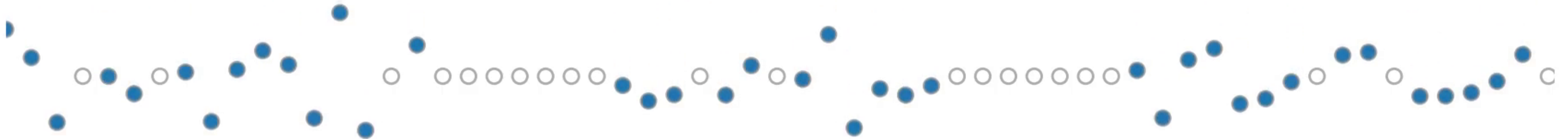
## GPU Optimisation: Light curve resampling



specify a regular time grid

# CETRA – Linear search

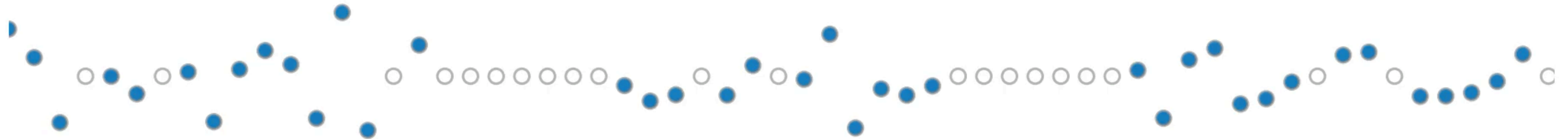
## GPU Optimisation: Light curve resampling



each point contributes to its nearest grid time, using inverse variance weighted means where necessary

# CETRA – Linear search

## GPU Optimisation: Light curve resampling



the two gaps are now stretches of null, zero-weight grid points

# CETRA — Linear search

## GPU Optimisation: Light curve resampling

Identifying which points on a regular time grid fall within a given transit window is trivial arithmetic — the GPU can do this very quickly

# CETRA — Linear search

## GPU Optimisation: Transit model lookup

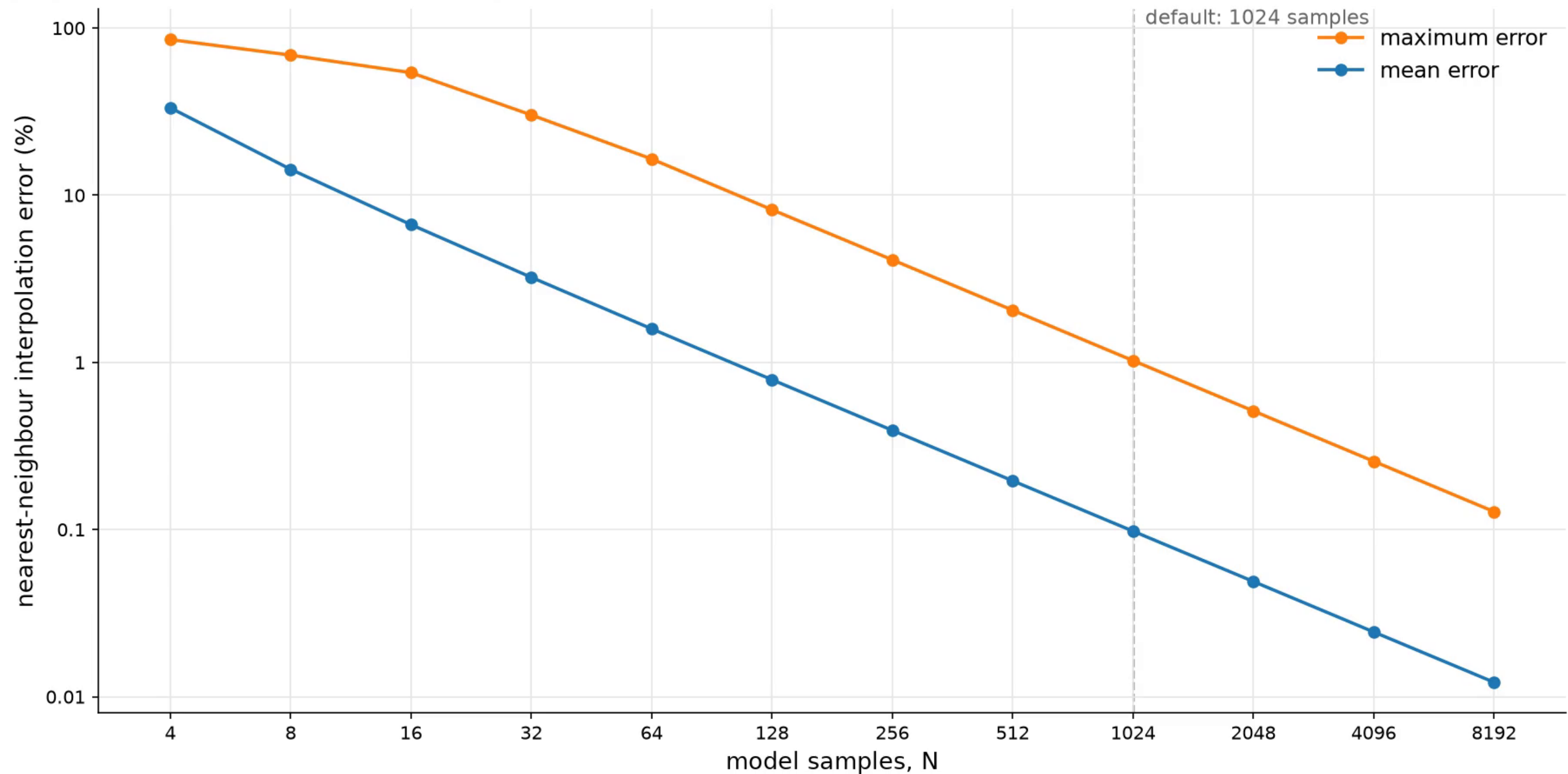
Rasterised transit model uses a simple nearest neighbour lookup for speed. The introduced error is easily quantifiable, and is acceptable.

# CETRA – Linear search

## GPU Optimisation: Transit model lookup

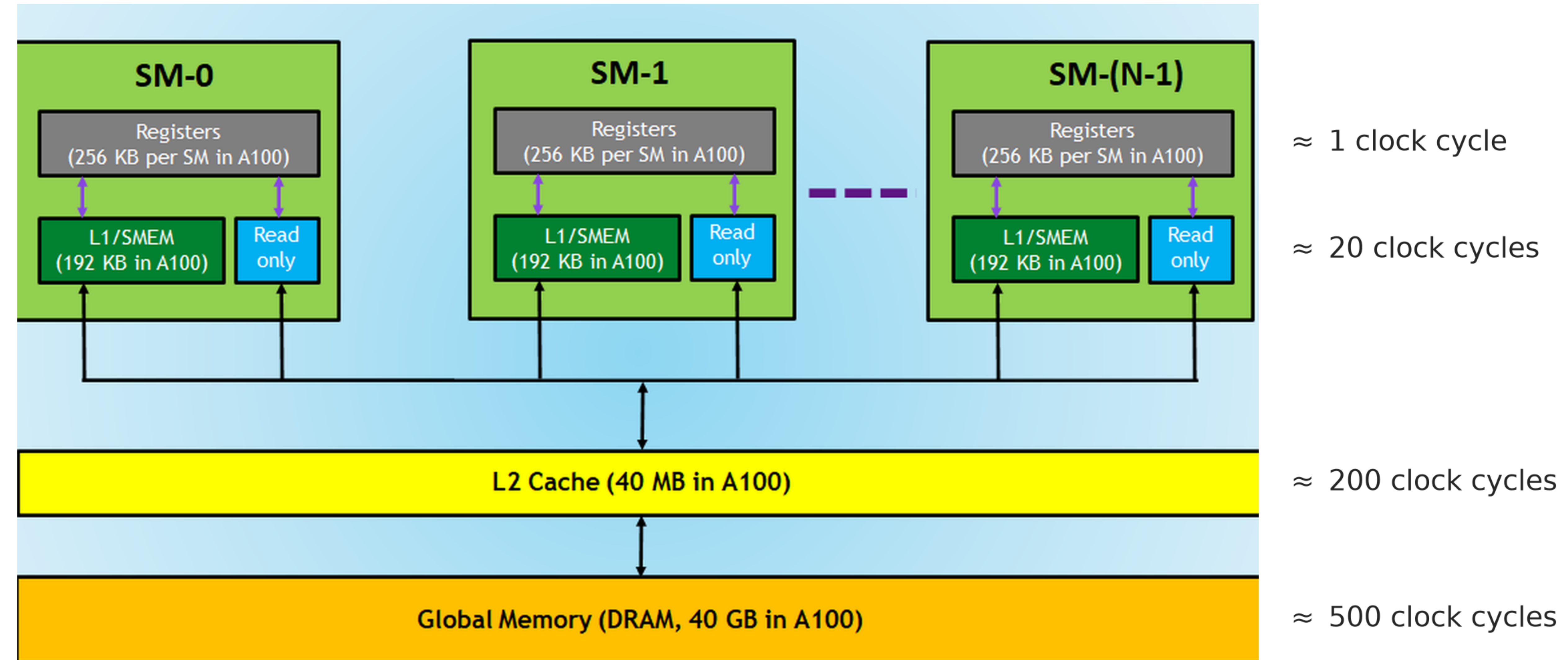
Nearest-neighbour interpolation error vs. model size

default (N = 1024): max error 1.0%, mean error 0.1%



# CETRA – Linear search

## GPU Optimisation: use of shared memory



# CETRA — Linear search

GPU Optimisation: use of shared memory

Transit model loaded into shared memory at the start of each thread

Partial sum arrays also stored in shared memory

— the many(!) accesses are much faster than global memory

# CETRA — Linear search

## GPU Optimisation: zero-indexed baseline flux

Typical representation is that normalised out of transit flux is 1.0:

At fp32 the floating point error is  $\approx 1$  ppm

$$1 \text{ ppm depth} = 1.000001 \times 10^0$$

Refactoring to 0.0 out of transit flux — simply subtract 1 (!!):

At fp32 the floating point error is essentially negligible

$$1 \text{ ppm depth} = 1.000000 \times 10^{-6}$$

# CETRA — Linear search

## GPU Optimisation: zero-indexed baseline flux

This enables use of single precision for most operations. For data centre grade hardware this is not so important, but for consumer or workstation grade GPUs it makes a huge difference.

# CETRA — Linear search

## GPU Optimisation: warp-sized blocks

One transit window per 32-thread block:

- Matches the hardware warp size — no wasted lanes, full warp occupancy
- Also neatly avoids shared memory bank conflicts during partial sum accumulation
- Blocks stride through trial  $t_0$  values for a given duration — the transit model loaded into shared memory at the start is reused for each, cutting global memory reads

# CETRA – Linear search

## GPU Optimisation: likelihood reframing

For any model, while assuming Gaussian measurement errors, the log-likelihood of the observed data is:

$$\ln p(\mathbf{f}|t_i, D_j, \Delta_{i,j}) = \sum_{n=1}^N \left( -\frac{1}{2} \frac{(\Delta_{i,j} m_n - f_n)^2}{\sigma_n^2} - \frac{1}{2} \ln (2\pi \sigma_n^2) \right)$$

where:

$\Delta_{i,j}$  is the maximum likelihood depth at the given duration and  $t_0$

$m_n$  is the model value at element  $n$

$f_n$  is the flux value at element  $n$

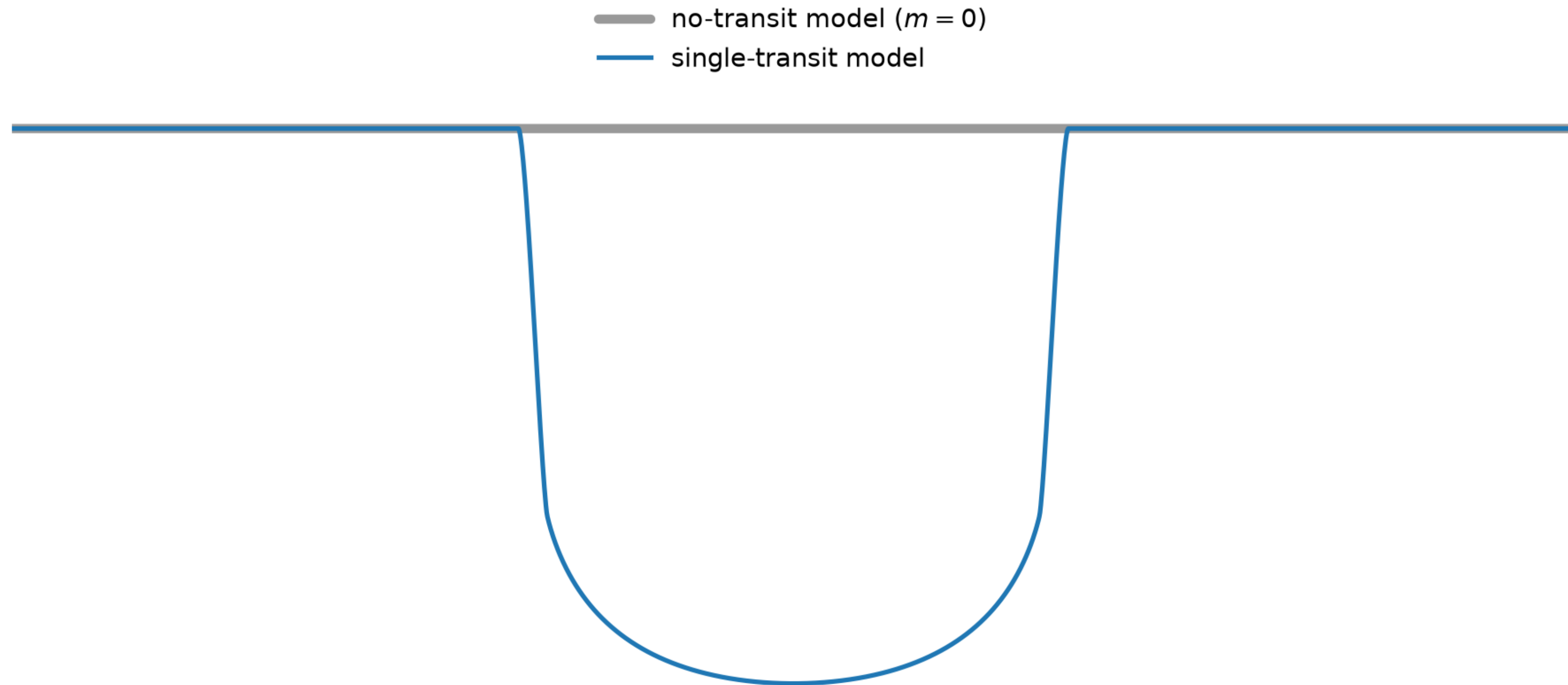
$\sigma_n$  is the flux error value at element  $n$

$N$  is the number of (resampled) light curve elements

# CETRA – Linear search

## GPU Optimisation: likelihood reframing

Note that out of transit flux is the same for both models...



# CETRA – Linear search

## GPU Optimisation: likelihood reframing

If we compute a *ratio* of likelihoods (transit vs. no transit) instead then all out of transit points cancel

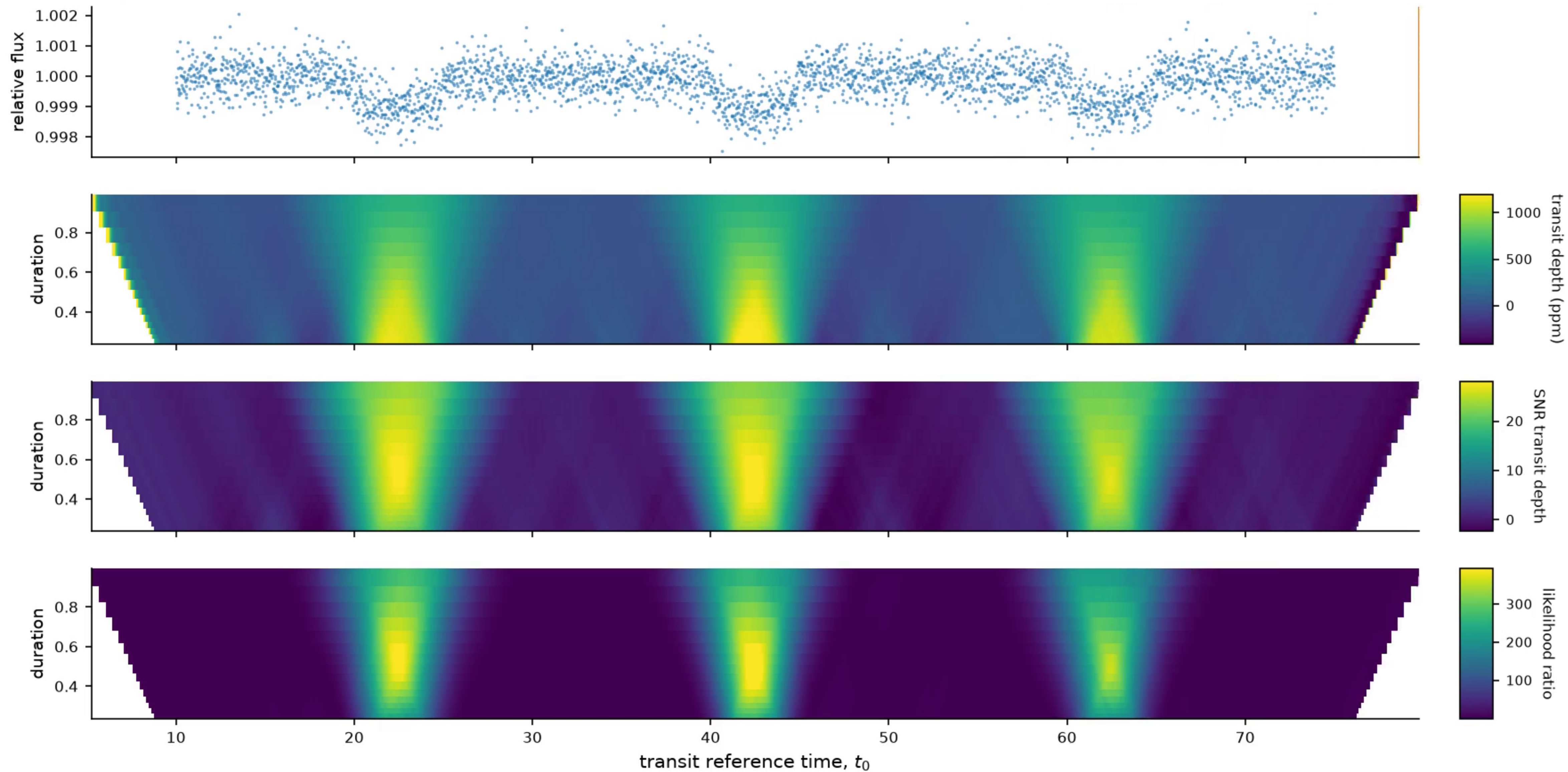
Also, the normalisation term is model-independent, so this cancels even inside the window

$$\ln \mathcal{L}_{\text{transit}} - \ln \mathcal{L}_{\text{no transit}} = \sum_{n \in \text{window}} \left( -\frac{(\Delta_{i,j} m_n - f_n)^2}{2\sigma_n^2} + \frac{f_n^2}{2\sigma_n^2} \right)$$

We avoid reading the vast majority of the light curve *and* a bunch of log operations in every loop

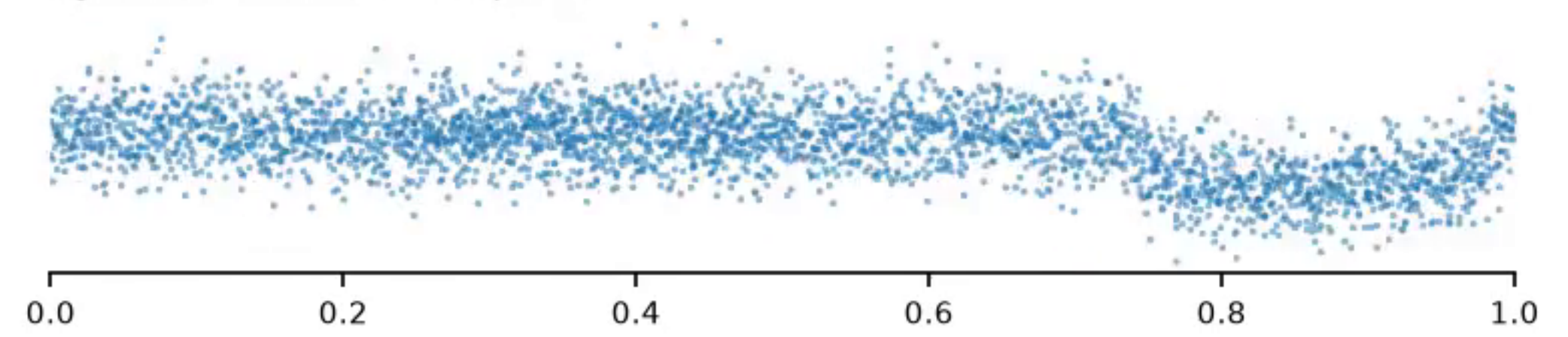
# CETRA — Linear search

done — the peaks in likelihood ratio mark the transits

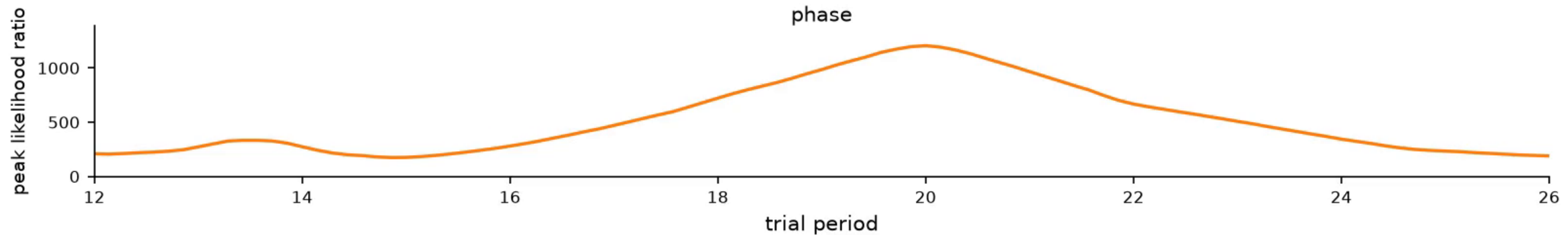
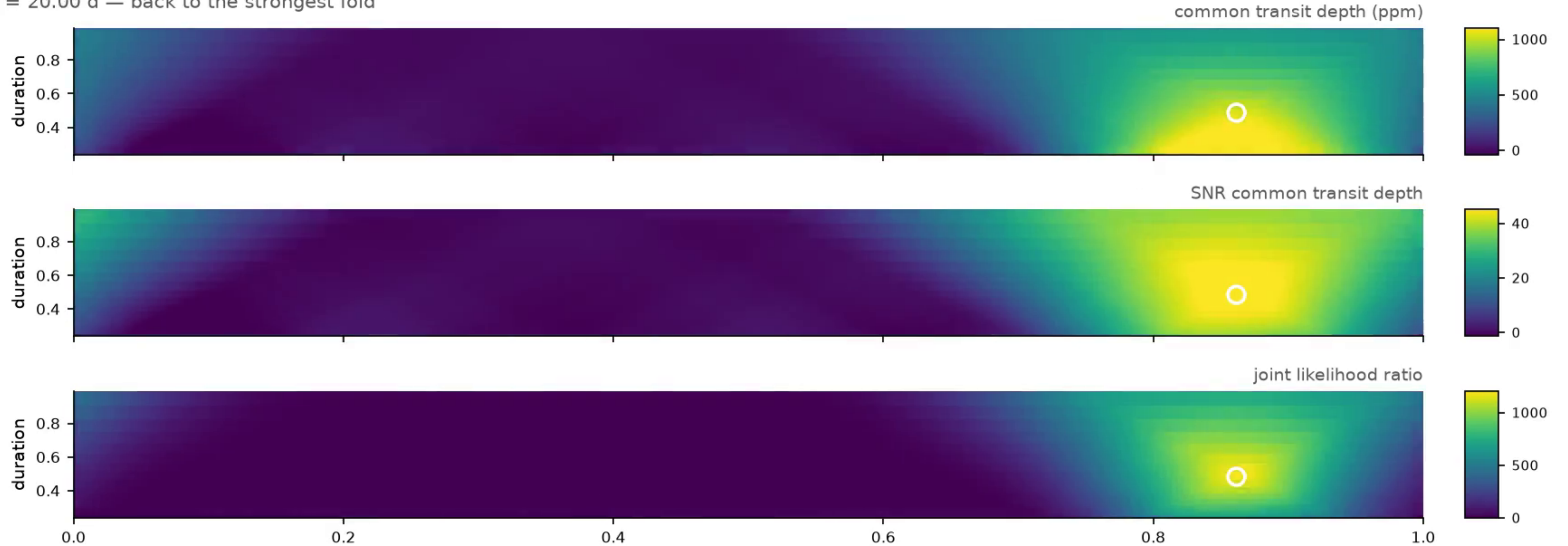


# CETRA — Period search

light curve, folded at trial period



P = 20.00 d — back to the strongest fold



# CETRA — Period search

## GPU Optimisations

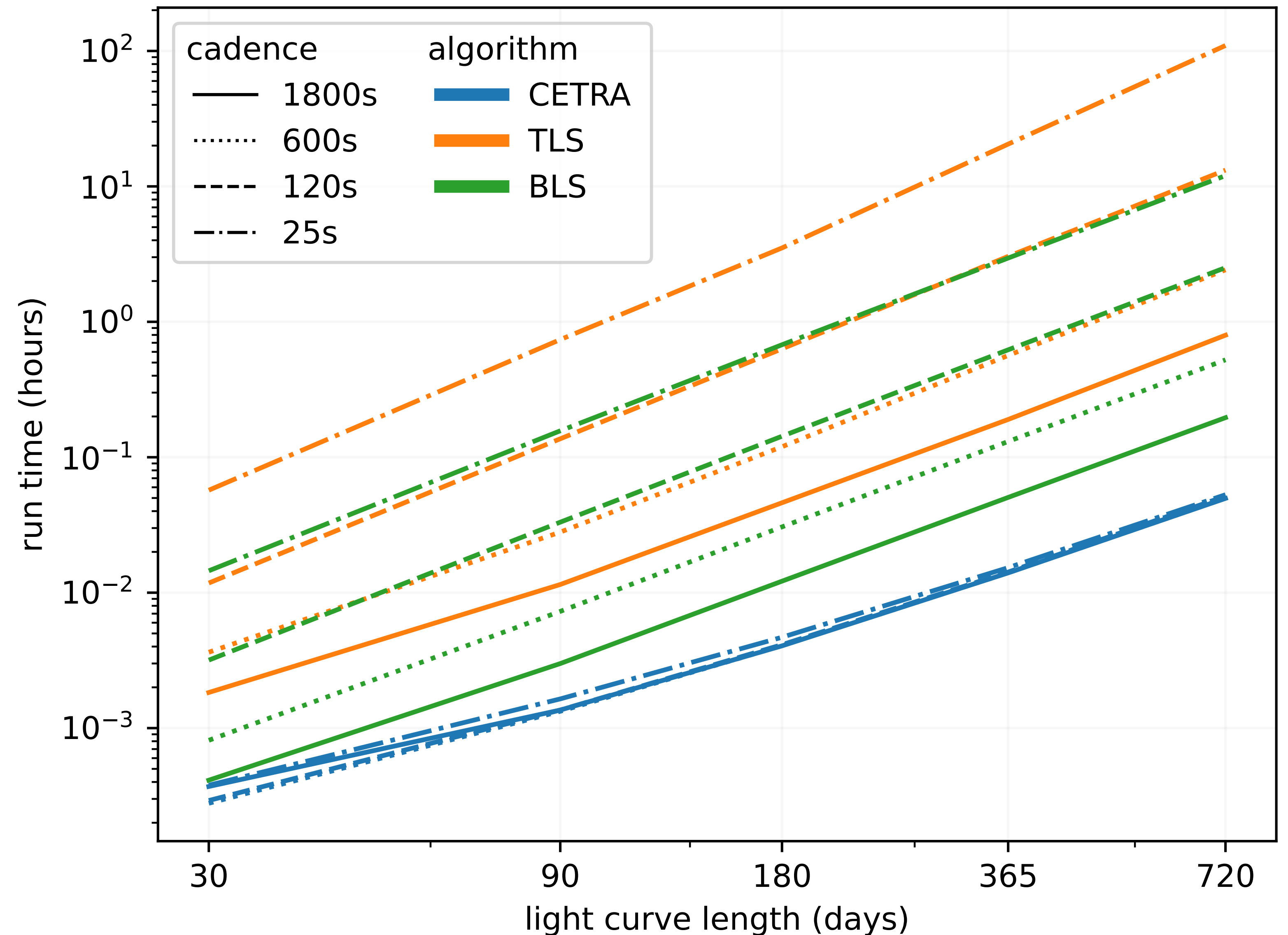
- Arrays already on device — no host → device transfer needed
- Regular  $t_0$  grid — identifying contributory elements is simple arithmetic
- `max()` reduction on device — only parameters of the strongest signal are returned for each period

# Performance

# CETRA performance — Compute speed

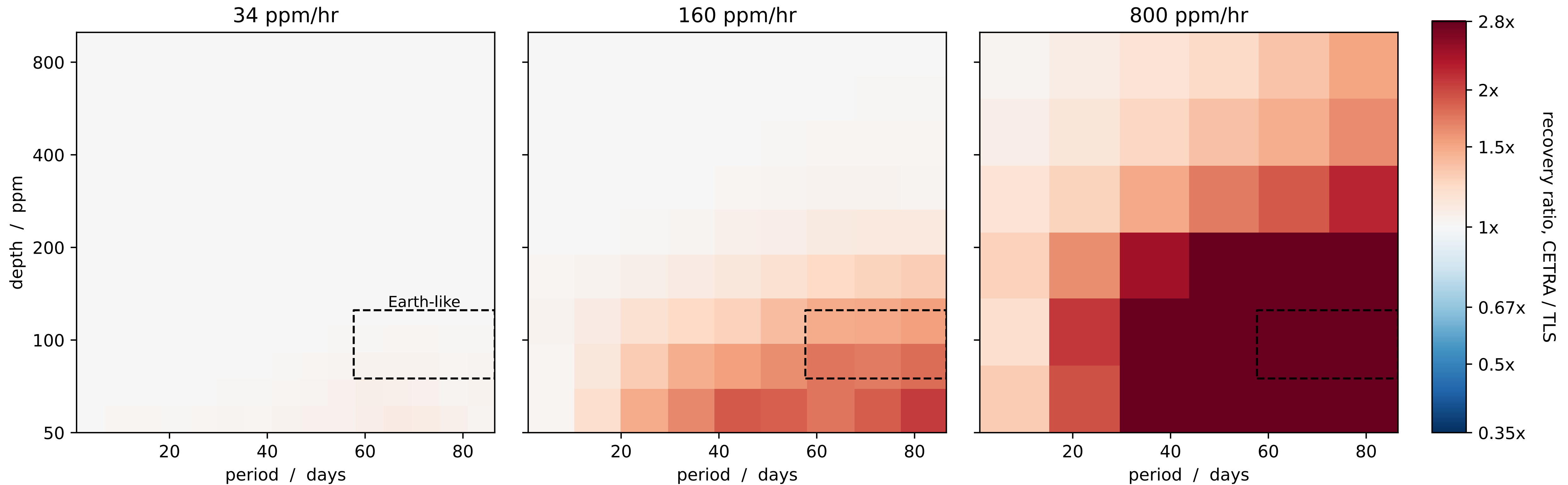
Run time of all 3 algorithms on either single CPU core or single GPU

- CETRA run time is cadence-independent
- Run time dominated by period search, which is cadence-independent



# CETRA performance — Sensitivity

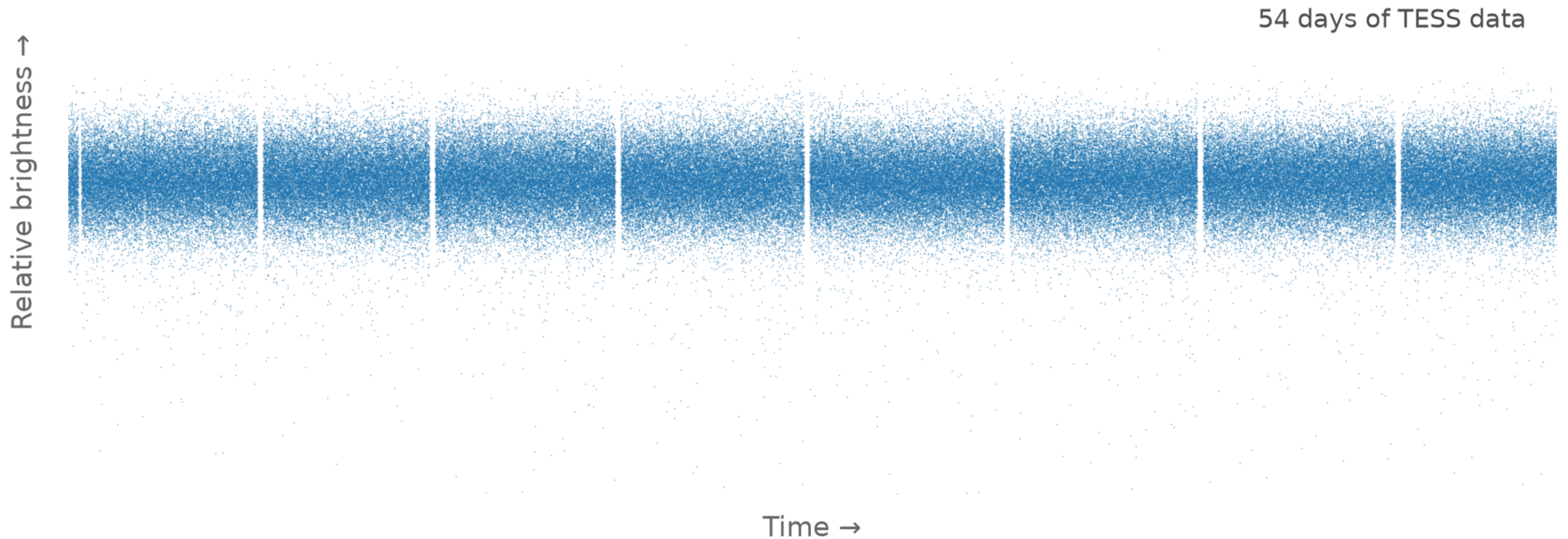
CETRA vs TLS recovery rate ratio (white-noise injections)



CETRA is more sensitive overall, particularly for low SNR signals

# CETRA — Finding real planets

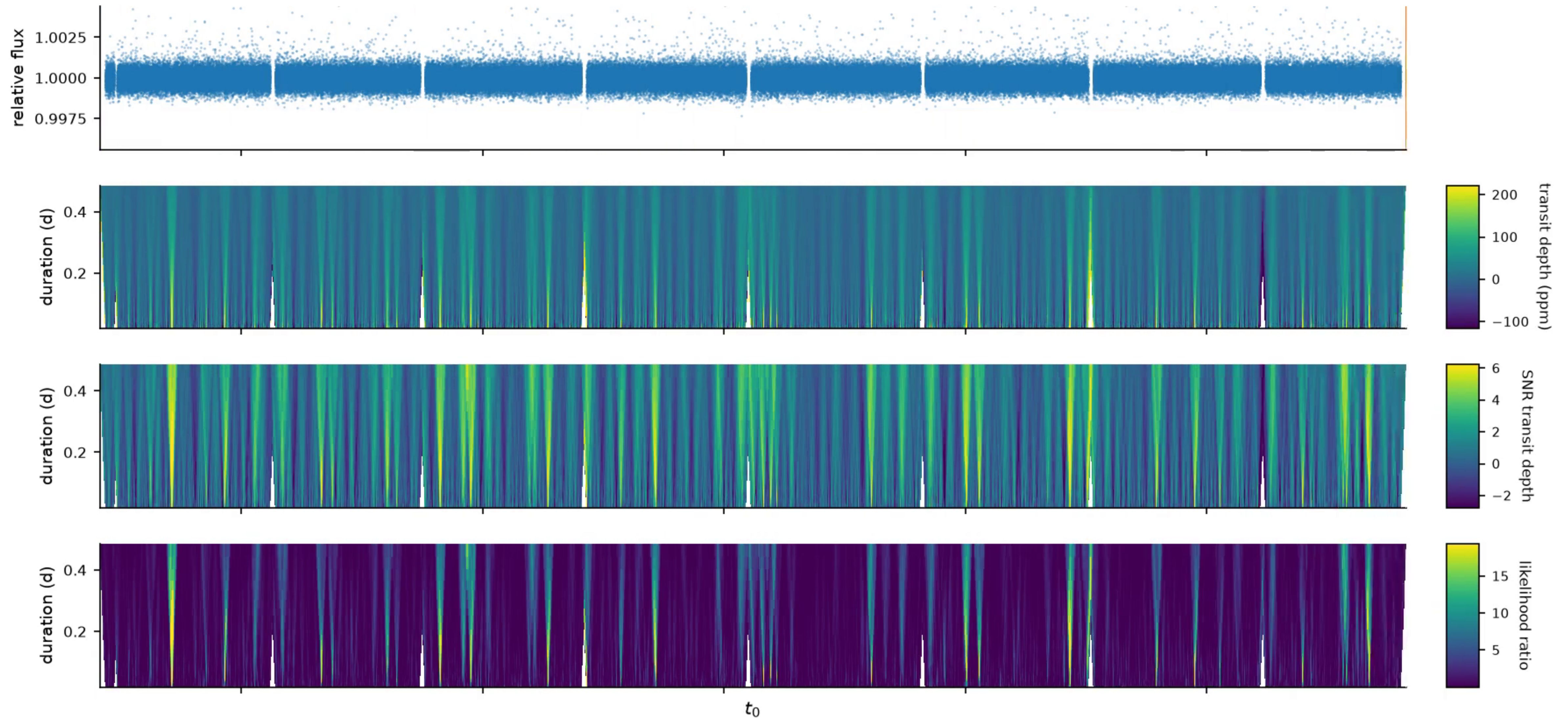
## HD 101581



# CETRA — Finding real planets

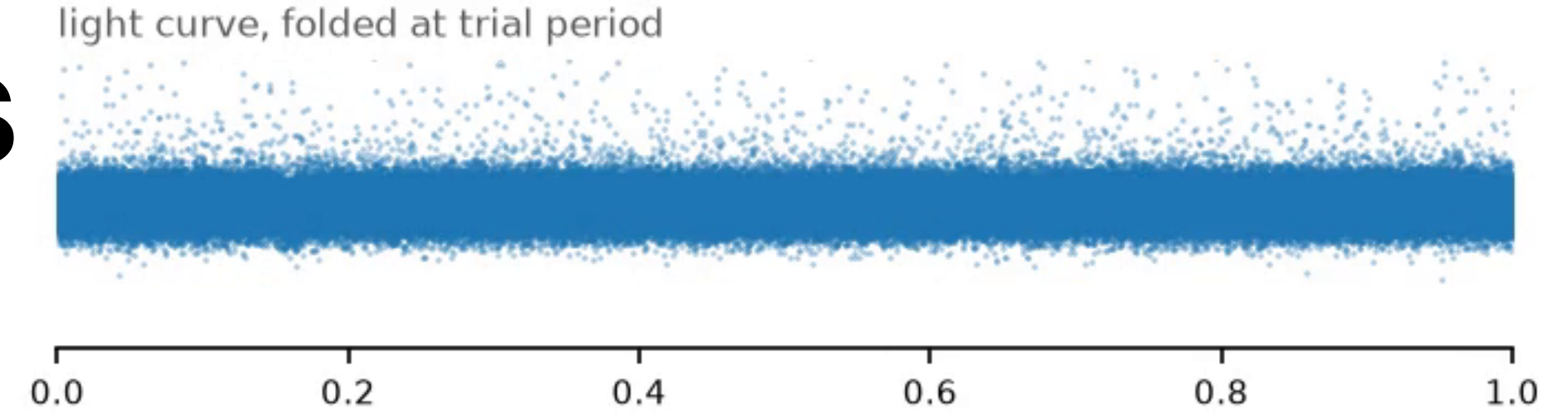
## HD 101581 — linear search

done — the peaks in likelihood ratio mark candidate transits

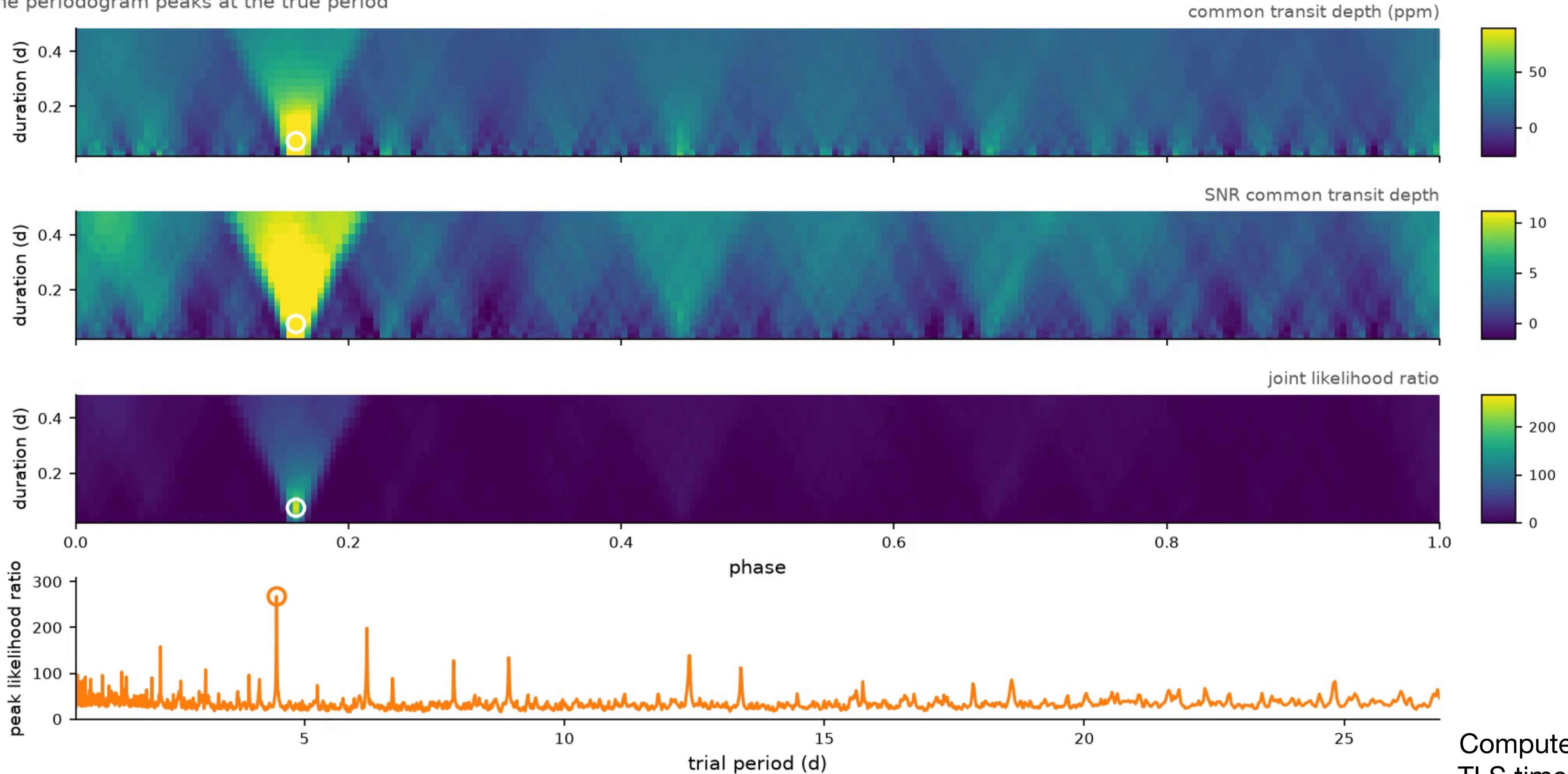


# CETRA — Finding real planets

## HD 101581 — period search



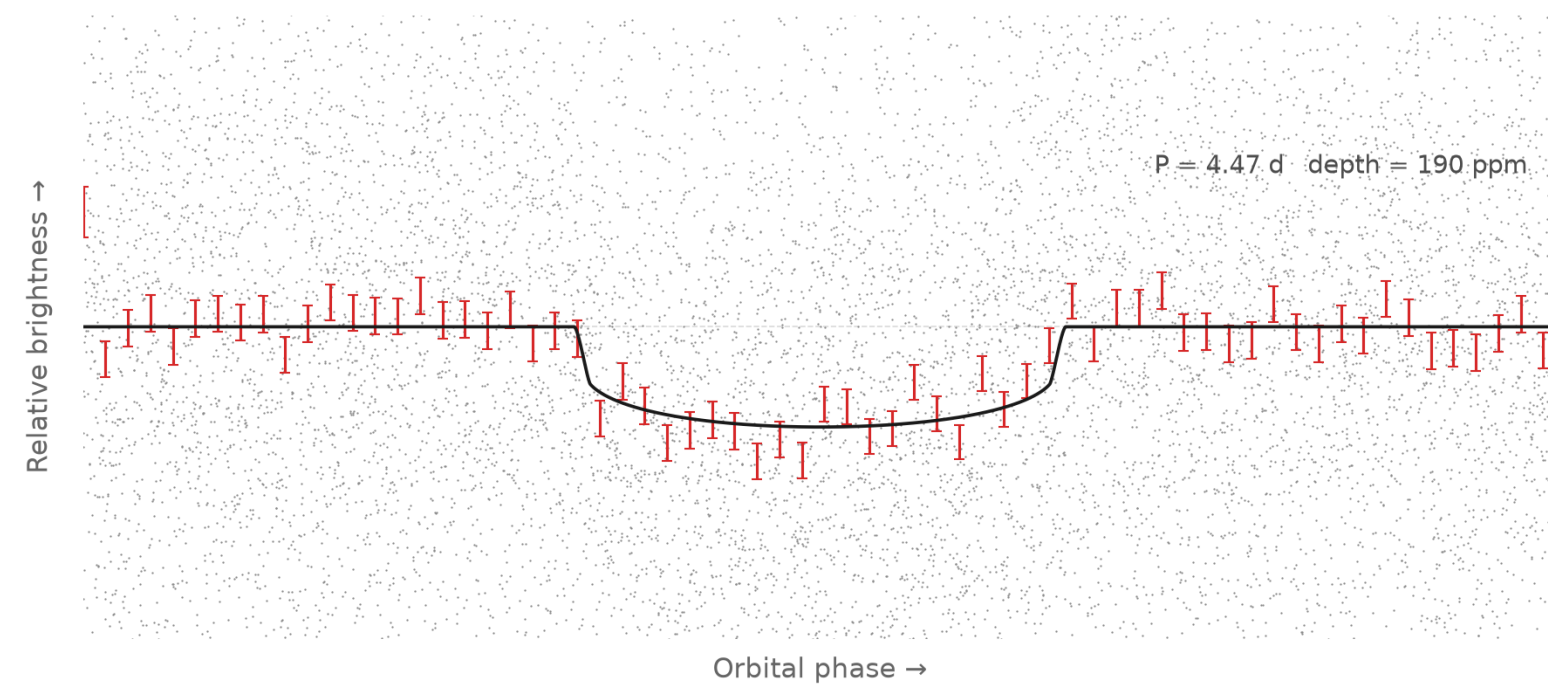
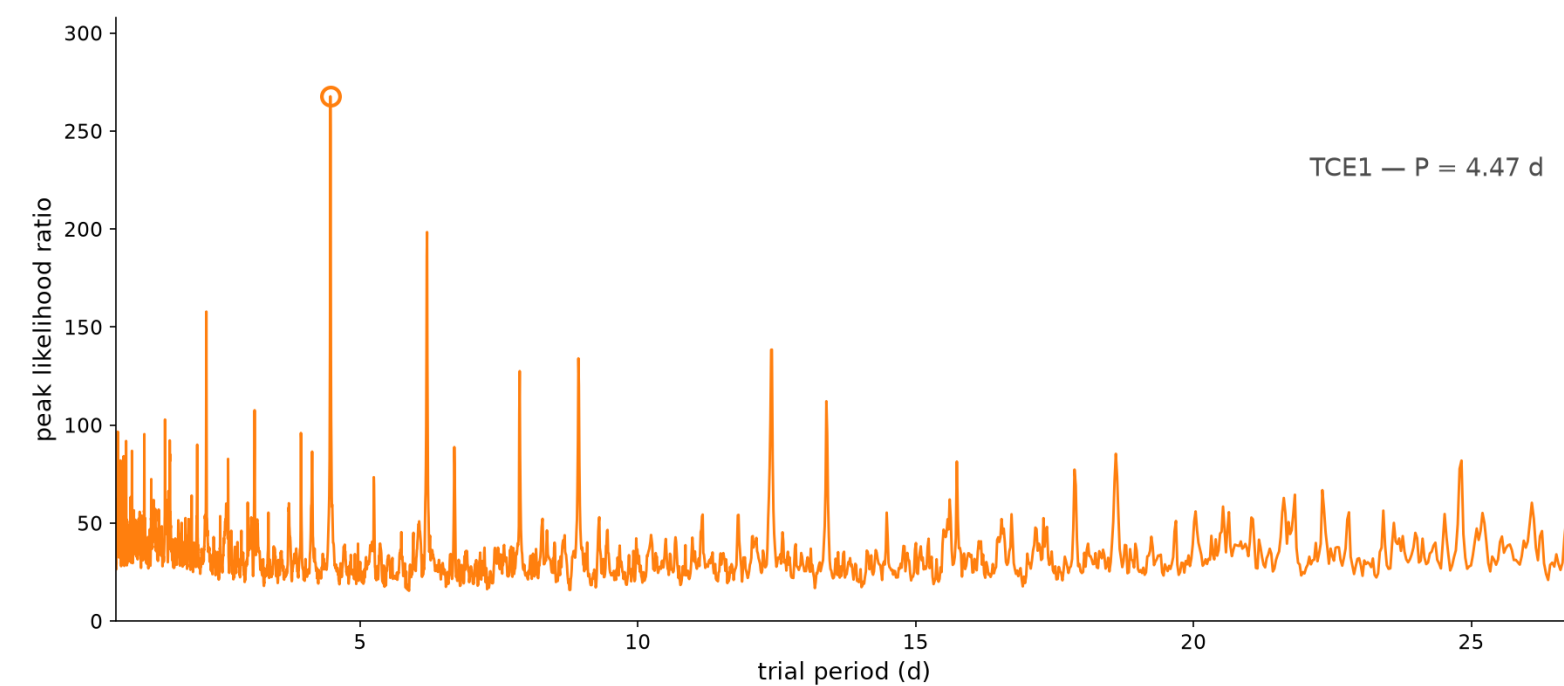
the periodogram peaks at the true period



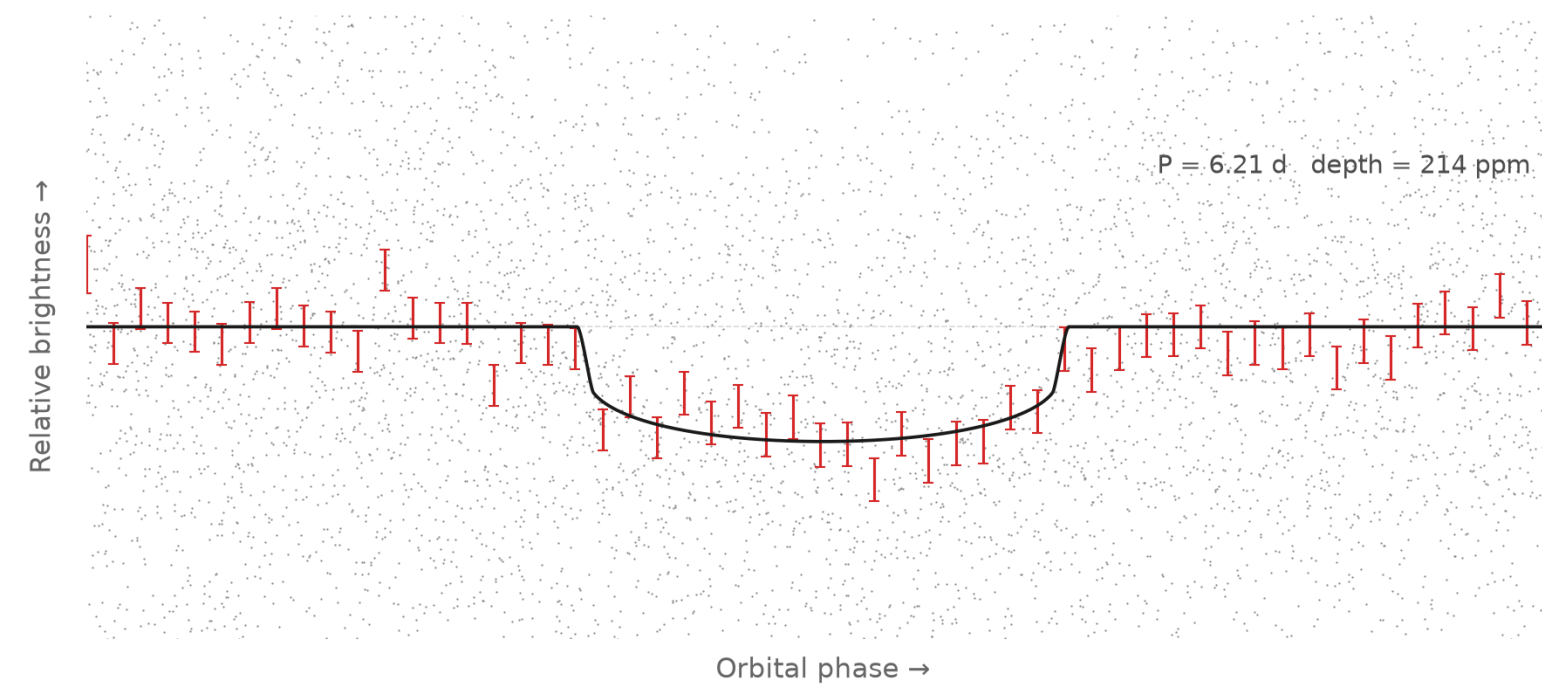
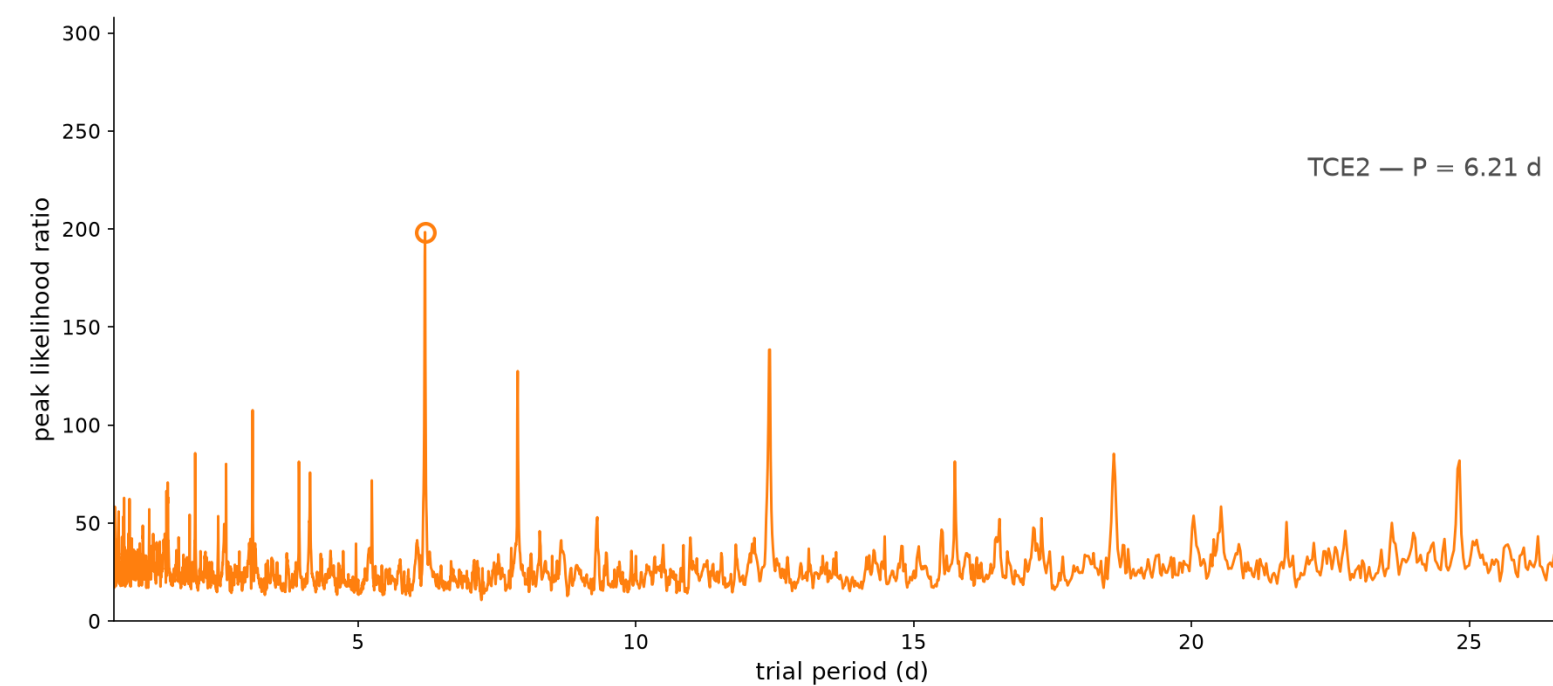
# CETRA — Finding real planets

## HD 101581 — masking and rerunning

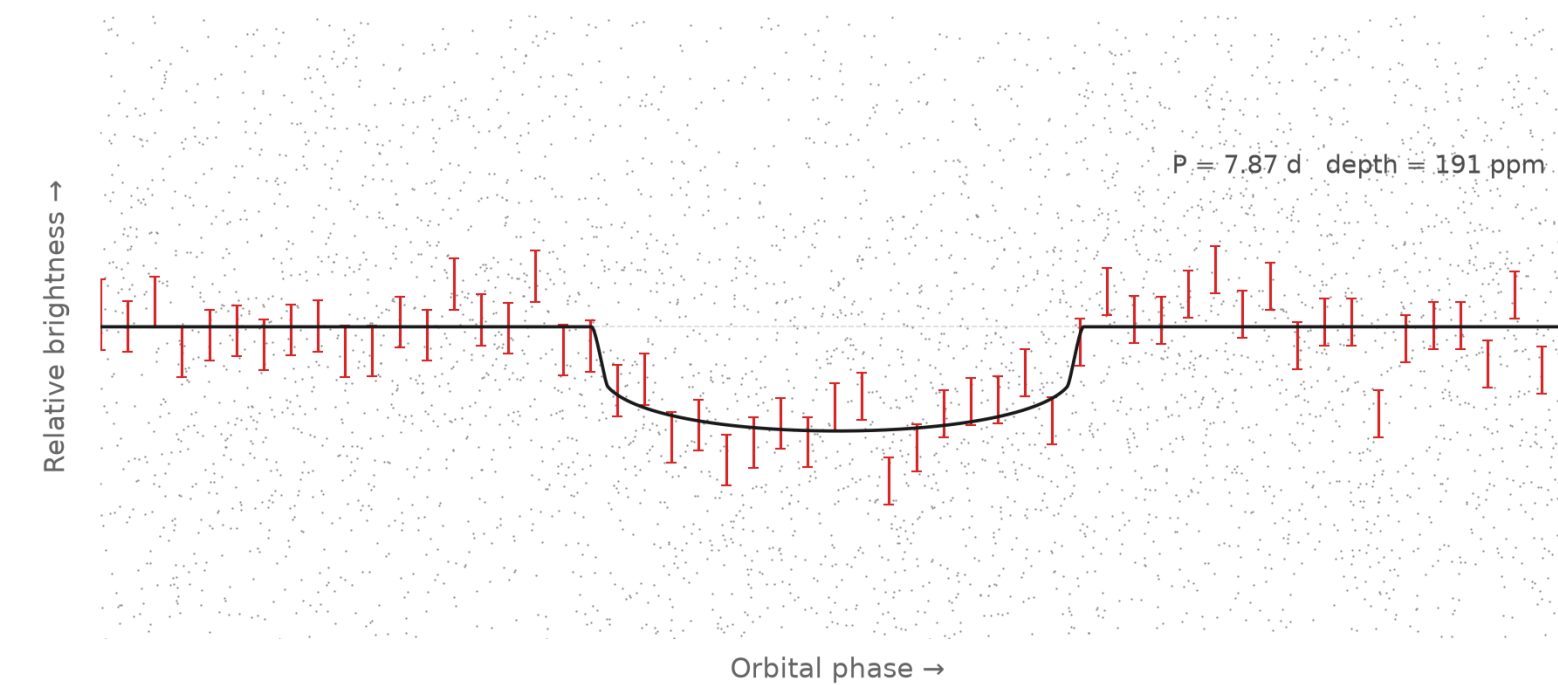
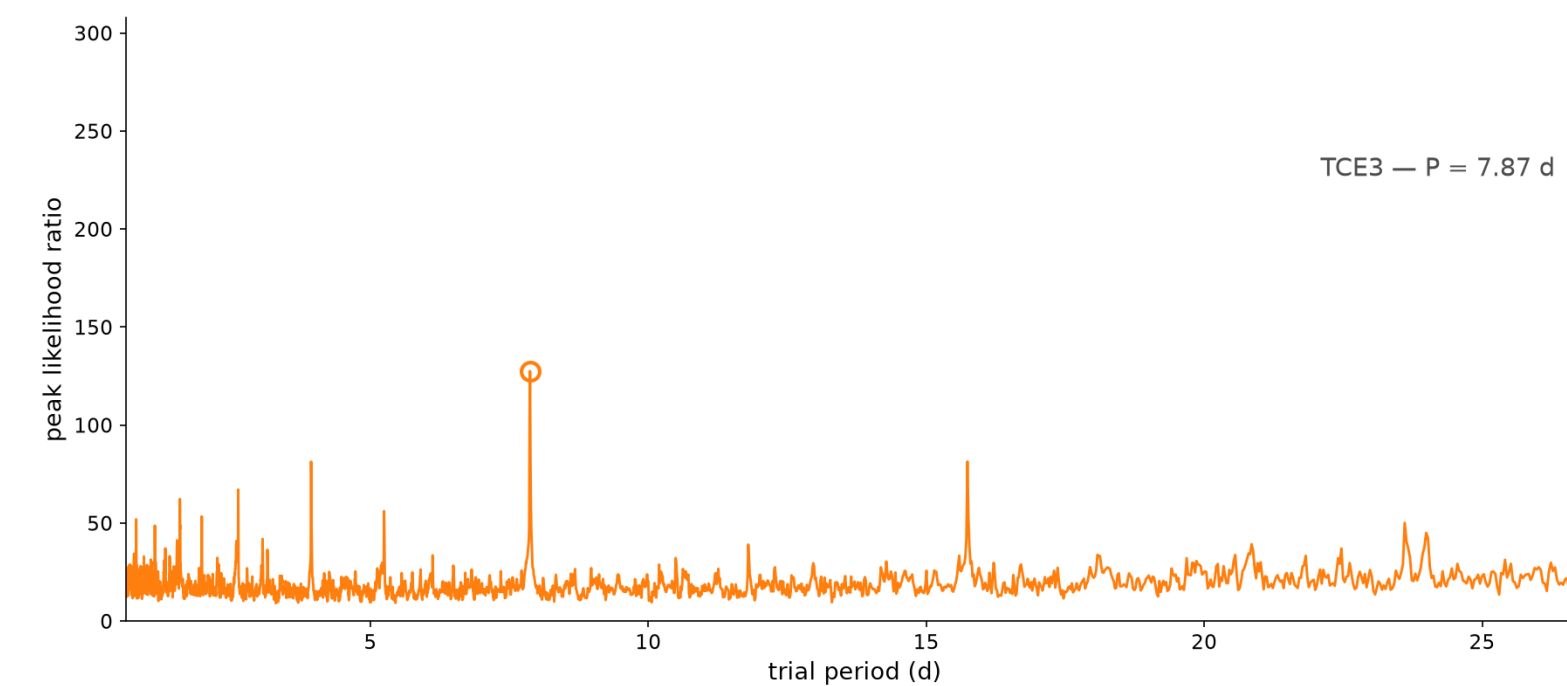
### planet 1



### planet 2



### planet 3



# Wrapping up

# Alternative frameworks

- **JAX**: designed around high-level array operations and auto-differentiation. Abstracts away some of the low-level control. (AMD support was not great in 2022 when I started writing CETRA)
- **OpenCL**: better choice for portability in hindsight. The CUDA → OpenCL transition isn't too hard, but CUDA documentation much better than OpenCL.
- Numba or CUDA offer the lowest barrier to entry

# CETRA — Resources

Smith et al. 2025, MNRAS 539, 297–306

[arXiv:2503.20875](#)

[github.com/leigh2/cetra](https://github.com/leigh2/cetra)

[cetra.readthedocs.io](https://cetra.readthedocs.io)

[lsmith.space](https://lsmith.space)

[lsmith@ast.cam.ac.uk](mailto:lsmith@ast.cam.ac.uk)